**Phuthiphong Wongchantib 660610840**

# Detour Steer — สรุป

อัลกอริทึมตัดสินใจของหุ่นที่เลี้ยวได้ทางเดียว: วิ่งไปก่อน เลี้ยวให้น้อยลง แล้วค่อยเข้าเป้า

## 1. ปัญหา

หุ่น `mor_luam` มี**มอเตอร์ตัวเดียว** หมุนทางหนึ่งเป็นการเลี้ยวล้อ (ได้แค่ทวนเข็มนาฬิกา) หมุนอีกทางเป็นการวิ่งไปข้างหน้า หุ่นจึงต้อง**หยุดก่อนเลี้ยว** และเลี้ยวขณะวิ่งไม่ได้ วิธีเดิมคือเลี้ยวตรงไปหาเป้าก่อนแล้วค่อยวิ่ง ถ้าเป้าอยู่ทาง**ขวา**นิดเดียว ($\varphi$ ใกล้ $360^\circ$) ล้อต้องหมุนเกือบรอบ

## 2. แนวคิด

เส้นตรงสองช่วงที่ทิศต่างกันเกิน $180^\circ$ ไปถึงได้ทุกทิศ แทนที่จะเลี้ยว $\varphi$ หุ่นเลือกได้ว่าจะ:

1. วิ่งตรงไปก่อน $a$ เมตร (ไม่เลี้ยว)
2. เลี้ยวแค่ $\beta^*$ $(180^\circ < \beta^* < \varphi)$
3. วิ่งอีก $b$ เมตรถึงเป้า

มุมที่ดีที่สุดมีสูตรปิด (ตัวเลข $k$ ตัวเดียวตัดสินทุกอย่าง):

$$k = \frac{d\,\omega}{v}\,|\sin\varphi|, \qquad \beta^* = 360^\circ - \arccos(k-1), \qquad a = \frac{d\,\sin(\beta^* - \varphi)}{\sin\beta^*}, \qquad b = \frac{d\,\sin\varphi}{\sin\beta^*}$$

$d$ = ระยะถึงเป้า, $\omega$ = ความเร็วเลี้ยว, $v$ = ความเร็ววิ่ง, $t_s$ = เวลารอล้อนิ่ง, $t_b$ = เวลาหยุดระหว่างสองช่วง

## 3. อัลกอริทึมตัดสินใจ

**Procedure :** PlanLeg($\varphi$, $d$) — // $\varphi$: มุมทวนเข็มไปหาเป้า, $d$: ระยะ

- $T_D \leftarrow \varphi/\omega + t_s + d/v$; — // วิธีเดิม: เลี้ยว $\varphi$ แล้ววิ่ง $d$
- **if** $\varphi \le 180^\circ$ **then return** DIRECT; — // T1: เลี้ยวตรงไปดีที่สุด
- $k \leftarrow (d\,\omega/v)\,|\sin\varphi|$;
- **if** $k \ge 2$ **then return** DIRECT; — // T3: อ้อมแบบไหนก็ไม่เร็วกว่า
- $\beta^* \leftarrow 360^\circ - \arccos(k-1)$; คำนวณ $a$, $b$;
- $T_R \leftarrow a/v + t_b + \beta^*/\omega + t_s + b/v$;
- **if** $\beta^* < \varphi$ **and** $T_R < T_D$ **then return** DETOUR($a$, $\beta^*$, $b$); — // อ้อมเฉพาะเมื่อเร็วกว่าจริง
- **return** DIRECT;

**end**

หุ่นเรียกฟังก์ชันนี้**ใหม่หลังขยับทุกครั้ง** (วงปิด) โดยใช้มุมล้อจริง (AS5600) และทิศตัวหุ่น (IMU) ตอนเลี้ยว หุ่นตั้งใจหยุดก่อนถึงมุมประมาณ $2^\circ$ เพราะขาดแค่เลี้ยวเพิ่มนิดเดียว แต่ถ้าเกินต้องหมุนเกือบรอบ ตอนวิ่ง หุ่นหยุดก่อนถึงแล้วเล็งใหม่ด้วยการเลี้ยวนิดเดียว ($\approx 18^\circ$)

## 4. ทำไมถูกต้องและเร็ว

- **T1** ถ้า $\varphi \le 180^\circ$ เลี้ยวตรงไปหาเป้าดีที่สุด
- **T2** ถ้า $\varphi > 180^\circ$ เส้นทางที่ดีที่สุดมีไม่เกิน 2 ช่วง
- **T3** เวลามีจุดต่ำสุดจุดเดียว ที่ $\cos\beta^* = k - 1$
- **T4** วางแผนใหม่หลังวิ่งช่วงแรก แผนยังเหมือนเดิม วงปิดจึงไม่แกว่ง
- **O(1)** ต่อการตัดสินใจหนึ่งครั้ง: 0.54 $\mu$s ใน Python เร็วกว่าการค้นหา 3600 จุด $944\times$ T1–T4 ตรวจแบบ brute force ใน `detour_steer.py` ผ่านทั้งหมด

## 5. ผลลัพธ์

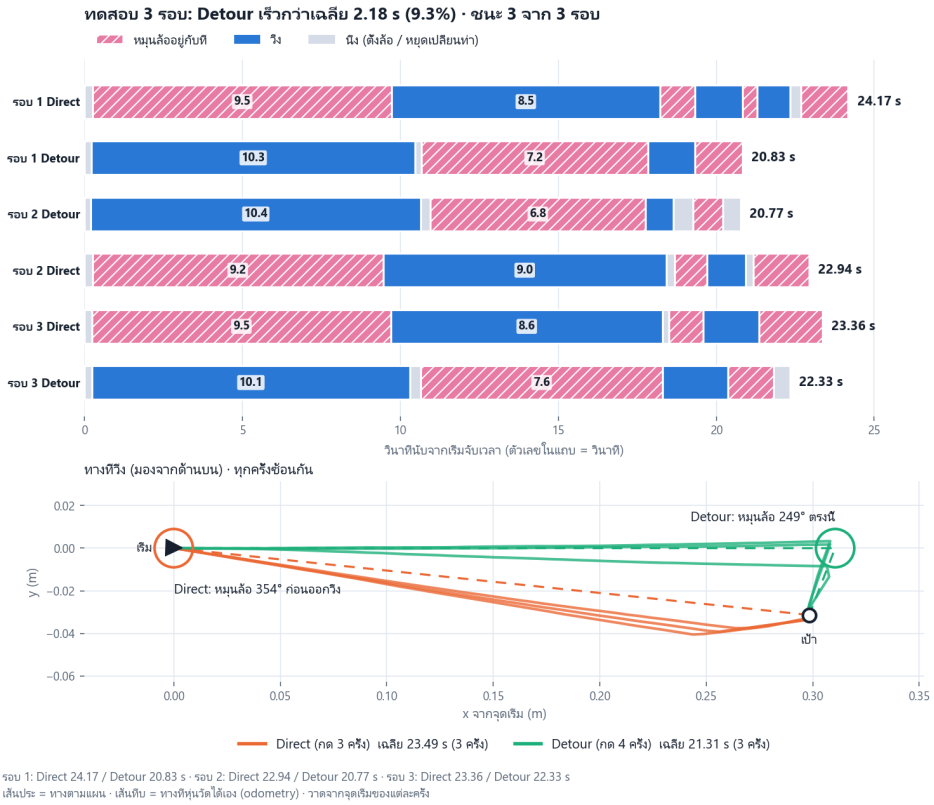
**ซิม** (เป้าสุ่ม 5000 จุด): Detour Steer ไม่ช้ากว่าวิธีเดิมเลย ประหยัด 0.4 % เมื่อล้อหยุดตรงพอดี, 2.3–6.9 % เมื่อล้อหยุดคลาดแบบสุ่ม และ 12.8–18.3 % เมื่อล้อมักหยุดเกินมุม

**บนหุ่นจริง** (เป้าห่าง 0.30 m ที่ $\varphi = 354^\circ$, $v = 0.03$ m/s, $\omega = 35^\circ$/s, 3 รอบ):

| รอบ | วิธีเดิม (s) | Detour Steer (s) | ประหยัด (s) |
|---|---|---|---|
| 1 | 24.17 | 20.83 | 3.34 |
| 2 | 22.94 | 20.77 | 2.17 |
| 3 | 23.36 | 22.33 | 1.03 |
| เฉลี่ย | 23.49 | 21.31 | 2.18 (9.3 %) |

Detour Steer ชนะทั้ง 3 รอบ เลี้ยวน้อยลง 3.8 s และวิ่งนานขึ้น 1.2 s ทั้ง 6 รอบหยุดห่างเป้าไม่เกิน 2.5 mm

**ทำไมออกขวา (ทั้งสองวิธี):** มอเตอร์ตัวเดียว จึงแก้ทิศระหว่างวิ่งไม่ได้, แรงขับหมุนชุดล้อและเฟืองไปเล็กน้อย (ระหว่างวิ่ง มุม AS5600 เปลี่ยนไปเรื่อย ๆ และหน้าหุ่นหมุน), ล้อลื่น พื้นไม่เรียบ และน้ำหนักไม่สมดุล, และล้อตั้งใจหยุดขาดมุมประมาณ $2^\circ$ ทำให้ชี้เยื้องขวาเล็กน้อย ทั้งสองวิธีออกขวา 1–3 cm แล้วการเล็งใหม่ก่อนถึงเป้า (เลี้ยวประมาณ $18^\circ$) ดึงกลับเข้าเป้า




รูปที่ 1 ผล 3 รอบจากข้อมูลที่หุ่นบันทึกเอง (บน: เวลาแต่ละช่วง, ล่าง: เส้นทางที่วัดได้)

## 6. สรุป

กฎสูตรปิดข้อเดียวตัดสินว่าจะ “เลี้ยวตรงไป” หรือ “วิ่งไปก่อน เลี้ยวให้น้อยลง แล้วค่อยเข้าเป้า” ใช้เวลา $O(1)$ ต่อการตัดสินใจ ไม่ช้ากว่าวิธีเดิมในซิม และเร็วขึ้น 9.3 % บนหุ่นจริง ยังไม่พบงานอื่นที่ให้สูตรปิดของการอ้อมแบบนี้ สำหรับหุ่นที่ต้องหยุดก่อนเลี้ยวและเลี้ยวได้ทางเดียว (“ไม่พบ” ไม่ได้พิสูจน์ว่าไม่มีใครเคยทำ)

## โค้ด Git และวิดีโอ

โค้ดทั้งหมด (ตัววางแผน Python, ซิม, เฟิร์มแวร์บนหุ่น) และรายงานฉบับเต็ม:

`https://github.com/Mannaja78520/mor_luam`

- ตัววางแผนและซิม: `algorithm_sim/` (รันได้โดยไม่ต้องใช้ Docker: `pip install matplotlib`)
- บนหุ่นจริง: `firmware/src/algorithm/DetourSteer.cpp`
- รายงานฉบับเต็ม: `algorithm_sim/docs/660610840_DetourSteer.pdf` (ไทย), `report.pdf` (อังกฤษ)
- วิธีติดตั้ง: `algorithm_sim/README.md`, `docs/INSTALL_TH.md` (แนะนำติดตั้งแบบ Docker)

**วิดีโอการทำงานของหุ่น:**

`https://drive.google.com/drive/folders/11qOM6UoeH2c4nEuM0a0JO6eXTnr0eDud`

คลิปนี้เป็นการทำงานของหุ่นแบบ synchronize ไม่มีคลิปของการเปรียบเทียบ Detour Steer กับวิธีเดิม (Direct) ผลการเปรียบเทียบดูได้จากข้อมูลที่หุ่นบันทึกเองในรูปที่ 1 และตารางผลบนหุ่นจริง